Describing a price line with splines: fixed knots and free knots

A plain-language but rigorous account of one experiment: fitting cubic splines to three share-price series, first with equally spaced knots and then with knots that are allowed to move. Written for a reader with no background in computer science, statistics or higher mathematics. Every number is taken from the saved experimental output; every formula is stated in full.

1. The question

Take the daily closing price of a share over three and a half years. It is a jagged line with about 900 points. Suppose you may describe that line using only a small list of numbers, say 15, or 60, or 240, and someone else must redraw the line from your list alone. How close can the redrawn line get? And how much closer does it get each time you are allowed more numbers? That is the whole experiment. In mathematics it is called **function approximation**: finding a simple formula whose graph stays close to a complicated one.

This is deliberately **not** forecasting. Nobody is asked what the price will be tomorrow. The entire series, past and future, is on the table from the start. The earlier part of this project showed that no model we tried could forecast prices better than simply repeating yesterday's price, because day-to-day price changes are close to unpredictable. Approximation sidesteps that: it asks only how well a given kind of formula can *describe* what has already happened, which is a question about the formula, not about the market.

Why it matters. If one kind of formula describes price paths far more compactly than another, that tells us something about the formula's expressive power that forecasting experiments cannot, because forecasting results are dominated by the unpredictability of the target rather than by the model. It also tells us how 'rough' price data is, which turns out to be the deepest finding here.

2. The data

Three shares, chosen to be as different as possible: Commonwealth Bank (CBA.AX, an Australian bank, the series the original paper used), Tesla (TSLA, very volatile) and Eli Lilly (LLY, a steady upward trend). Daily closing prices from 2 January 2020 to 31 July 2023: 905 trading days for CBA, 901 for the two US shares.

Two rescalings make the three series comparable. Time is rescaled so that the first day is t = 0 and the last day is t = 1. Price is rescaled so that the lowest close in the period is 0 and the highest is 1 (this is called Min-Max scaling: p_scaled = (p - p_min) / (p_max - p_min)). After this, every series is a set of about 900 points (t_i, p_i) with both coordinates between 0 and 1. An error of 0.01 therefore means one hundredth of the share's whole price range over the period.

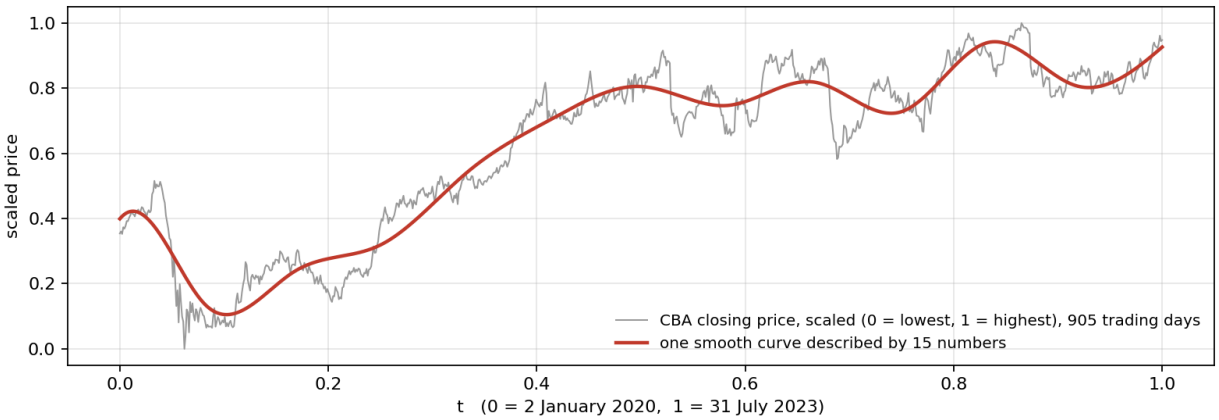


Figure 1. The CBA series after rescaling (grey), and one smooth curve that tries to follow it using only 15 numbers (red). The curve catches the broad movements and misses the fine ones. Everything in this document is about making curves like the red one, with different kinds of formula and different numbers of parameters.

3. The mathematics we need

3.1 Functions and parameters

A **function** is a rule that turns an input into an output. Here the input is a time t between 0 and 1 and the output is a price. We write f(t). A **family** of functions is a formula with blanks in it; the numbers that fill the blanks are the **parameters**. For example f(t) = a + b t + c t^2 is a family with three parameters a, b, c; choosing values for them picks one parabola out of the family. 'Describing the price line with 60 numbers' means picking the member of a 60-parameter family that sits closest to the data.

3.2 Measuring closeness: the error and the RMSE

For each day i we have the real price p_i and the curve's value f(t_i). Their difference e_i = f(t_i) - p_i is that day's **error**: positive if the curve is above the price, negative if below. One number summarises all the errors, the **root-mean-square error**:

```
RMSE = sqrt( (1/n) * sum over i of e_i^2 ) (n = number of days)
```

In words: square every error (so positives and negatives do not cancel and big errors count more), average the squares, take the square root to get back to price units. A second number, the **maximum absolute error** max |e_i|, records the single worst day. Both are reported for every fit.

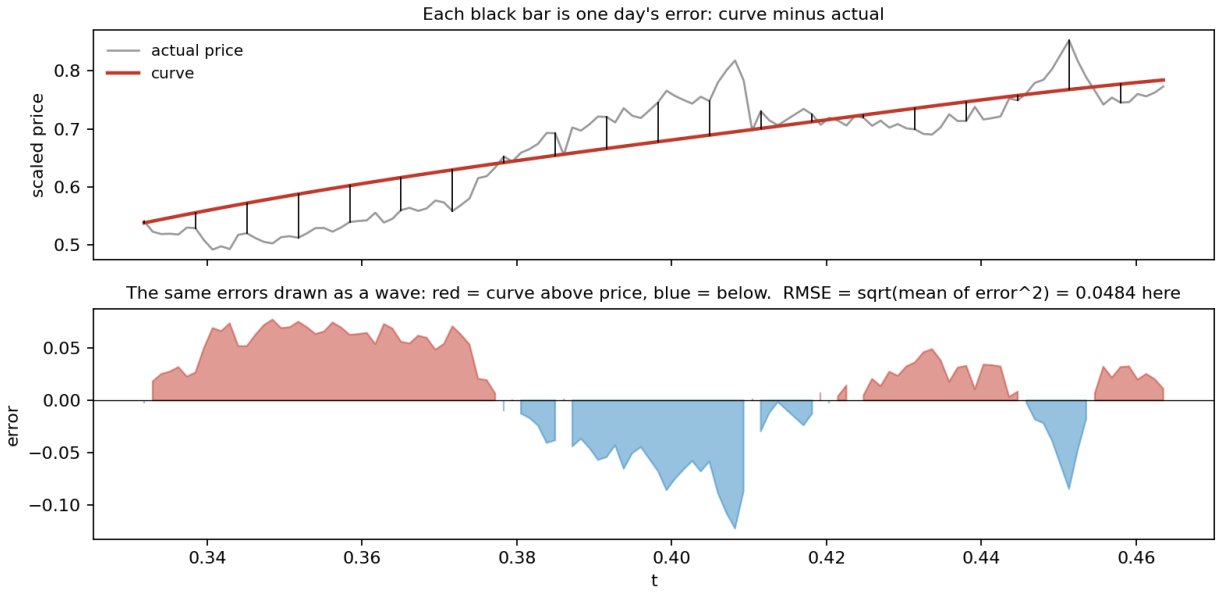


Figure 2. Top: a stretch of the price (grey) and a curve (red); each black bar is one day's error. Bottom: the same errors drawn as a wave, red where the curve is above the price, blue where below. The RMSE is the typical height of this wave. The result PDFs draw this wave for every fit, with a black envelope showing the 20-day rolling RMSE, i.e. the local typical error with the day-to-day jitter smoothed out.

3.3 'Least squares'

Among all members of a family, the one we want is the one with the smallest RMSE. Because the RMSE is built from squared errors, this is called the **least-squares fit**. For some families (splines) the least-squares fit can be computed exactly by solving a system of linear equations; for others (neural networks) it must be searched for by trial and improvement, which is what 'training' means, and the search may stop at a good-but-not-best answer.

3.4 Comparing fairly: parameter budgets

A formula with more parameters can always fit at least as well, so comparing two formulas is only meaningful at the **same number of parameters**. We therefore fixed five budgets, about 15, 30, 60, 120 and 240 parameters, set each model up to use as close to the budget as its structure allows, and recorded the exact count actually used. The plots put the true count on the horizontal axis.

3.5 Rate of improvement, and why we use logarithmic axes

The most informative quantity is not any single error but how fast the error shrinks as the parameter count N grows. For many approximation methods the error follows a **power law**, error = C * N^(-a), where a is the **rate**. Taking logarithms turns this into log(error) = log C - a * log N, a straight line whose slope is -a. So on axes where both the parameter count and the error are drawn logarithmically, a straight line means a power law and its steepness is the rate: slope -1 means doubling the parameters halves the error; slope -0.5 means doubling the parameters only cuts the error by a factor of 1.41 (the square root of 2). We fit this slope to each model's results.

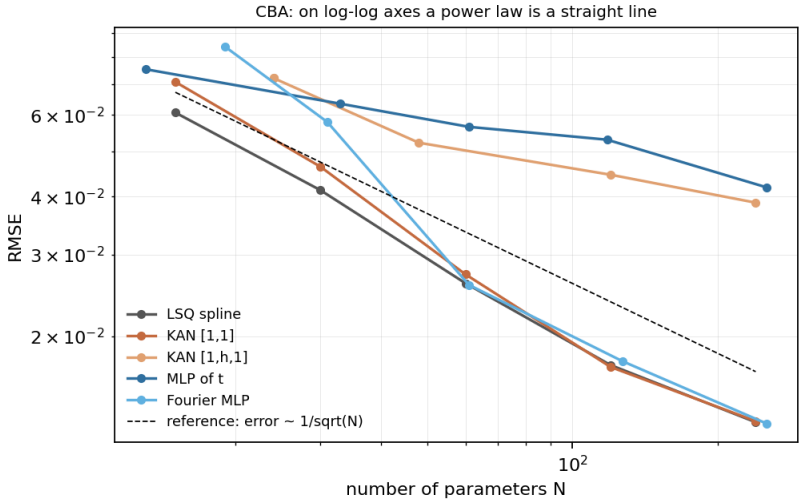


Figure 3. The CBA results on logarithmic axes. Each model's points lie roughly on a straight line; the dashed reference line has slope -0.5 (error proportional to 1 / sqrt(N)). The slope of each model is listed in the results section.

3.6 How rough is a price line? (the key background fact)

Zoom into a smooth curve and it eventually looks like a straight line. Zoom into a price line and it never does: at every magnification it is still jagged (Figure 4). Mathematicians measure this with the **Holder exponent**: a function is Holder-continuous of order alpha if |f(x) - f(y)| is at most a constant times |x - y|^alpha for all nearby x, y. A smooth curve has alpha = 1 (it is 'Lipschitz'); the path of a random walk, which is the standard model of daily prices, has alpha just below 1/2 and no higher (this is a theorem about Brownian motion; see Morters and Peres, *Brownian Motion*, Chapter 1). Classical approximation theory (de Boor, *A Practical Guide to Splines*) then says that any piecewise-smooth approximation with N pieces of a Holder-alpha function has error of order N^(-alpha). For a price path that predicts a rate of about 0.5: **doubling the parameters can only reduce the error by about 30 %, whatever the architecture**. Keep this prediction in mind; the results test it.

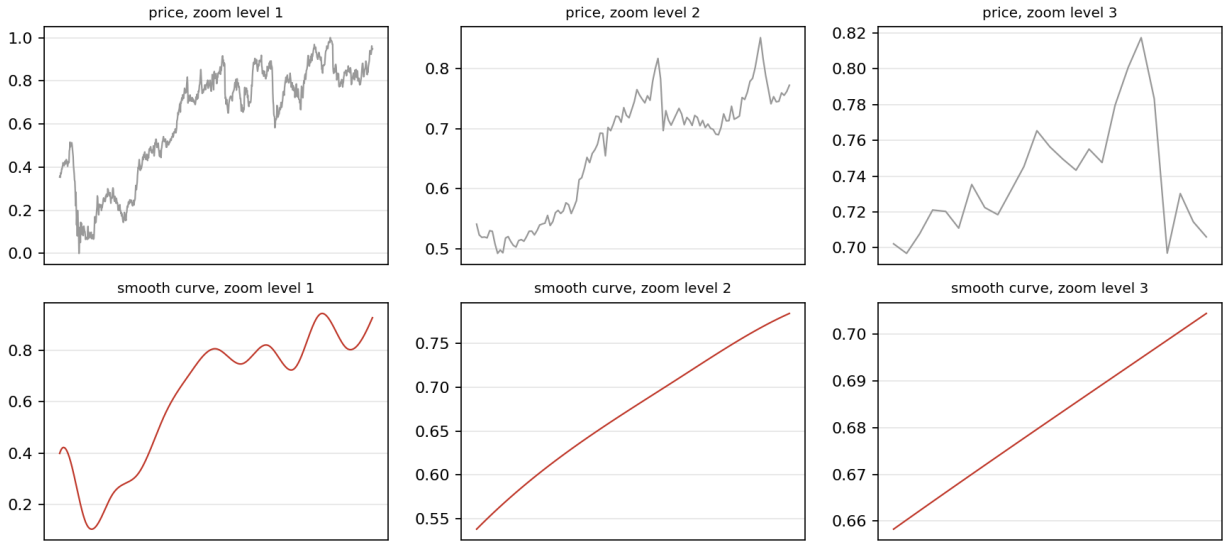


Figure 4. Top row: the CBA price line at three magnifications; it stays jagged. Bottom row: a smooth curve at the same magnifications; it straightens out. This difference is what limits every approximator in this study.

4. The two models

4.1 What a spline is

A **spline** is a curve made of polynomial pieces joined end to end. Split the time axis [0, 1] into G equal **intervals**; the G - 1 interior boundary points are the **knots**. On each interval the curve is a cubic polynomial (a + b t + c t^2 + d t^3), and at every knot the pieces are joined so that the curve, its slope and its curvature all match (in symbols: f, f' and f'' are continuous). That smoothness at the joins is what makes a spline look like one curve rather than a chain of segments.

Rather than writing the polynomial of each piece separately, a cubic spline is written as a weighted sum of fixed 'bump' functions called **B-splines**: f(t) = sum over j of c_j B_j(t). There are G + 3 bumps for G intervals, each bump is non-zero on only four neighbouring intervals, and the bumps are the same for every data set; only the weights c_j (the **coefficients**) are fitted. The coefficients are the model's parameters: a fixed-knot cubic spline on G intervals has exactly G + 3 of them.

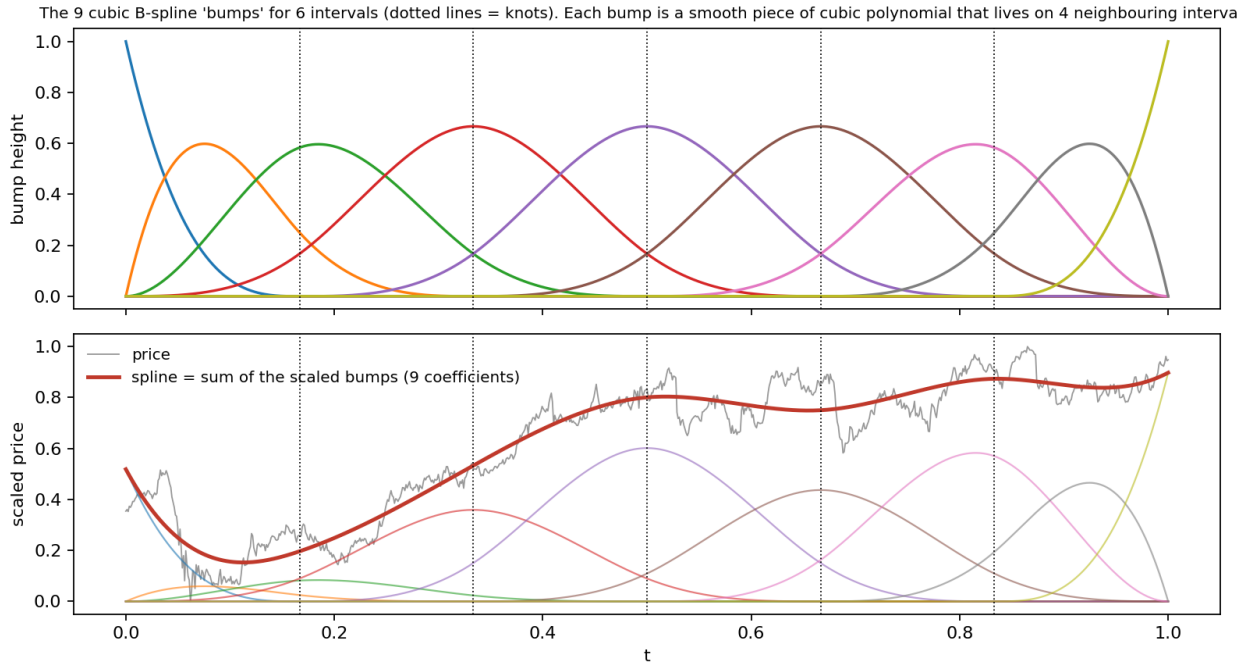


Figure 5. Top: the nine B-spline bumps for six intervals (knots dotted). Bottom: scaling each bump by its fitted coefficient and adding them up gives the spline (red) that best follows the CBA series with nine numbers.

Because f(t) is a *linear* combination of known bumps, the least-squares coefficients are found exactly by solving G + 3 linear equations; there is no searching and no randomness. This is why the spline rows in the results have no '+-' spread: the same input always gives the same answer.

4.2 Model A: least-squares cubic spline with fixed, equally spaced knots

Knots at t = 1/G, 2/G, ..., (G-1)/G. Parameters = G + 3 coefficients. Budgets 15, 30, 60, 120, 240 use G = 12, 27, 57, 117, 237 intervals. Implementation: scipy.interpolate.LSQUnivariateSpline with k = 3.

4.3 Model B: free-knot cubic spline

Identical, except that the G - 1 interior knot positions are themselves chosen to minimise the RMSE. Each knot position is one more parameter, so the total is (G + 3) + (G - 1) = 2G + 2. To hit the same budgets the free-knot spline must use fewer intervals: G = 7, 15, 30, 60, 120 (giving 16, 32, 62, 122, 242 parameters).

Finding the best knot positions is a genuinely hard search: the RMSE as a function of the knot positions has many local minima, and the usual linear-equation trick no longer applies. We used 12 random starting arrangements, each improved first by a gradient method (L-BFGS-B) and then by a derivative-free method (Nelder-Mead), with the coefficients re-solved exactly by least squares every time the knots move, and kept the best of the 12. Knots were kept at least four data points apart so that the linear equations stay solvable.

4.4 The one thing that differs

Whether the knots may move. The hope behind free knots is that a price line has a few 'important moments' (a crash, a turning point) where a knot is worth far more than elsewhere, so that cleverly placed knots beat many evenly spaced ones. Figure 6 shows the idea on a toy function with exactly that structure.



Figure 6. A piecewise-linear toy function with kinks at 3, 5, 6, 7 (grey). Left: a linear spline with knots at 2, 4, 6, 8 cannot reproduce it (max error 1.76). Right: with knots moved to the kinks the reproduction is exact. Free knots are powerful when the function has a few special places. The question is whether a price line is like that.

5. The experimental setup

For each of the three shares, each model and each of the five parameter budgets, the model was fitted to the whole series by least squares (exactly for splines, by numerical optimisation for the networks). Models whose fitting involves randomness (random starting values, random frequencies) were run three times with different random seeds (0, 1, 2) and the mean and spread are reported. In total 210 fits. For every fit we saved the fitted value on every day, the RMSE, the maximum error, the parameter count, the wall-clock time and the settings.

Settings per budget for the two models in this document:

budget	15	30	60	120	240
fixed-knot: intervals G / parameters	12 / 15	27 / 30	57 / 60	117 / 120	237 / 240
free-knot: intervals G / parameters	7 / 16	15 / 32	30 / 62	60 / 122	120 / 242

Software: Python 3.10, NumPy/SciPy for splines, pykan 0.2.8 for KANs, PyTorch 2.2 for MLPs; single CPU. Code: approx_study.py (fits), aggregate_approx.py (tables and figures), build_pdfs.py (result PDFs).

6. Results

share	budget	fixed-knot RMSE	max err	params	free-knot RMSE	max err	params
CBA.AX	15	0.0607	0.2258	15	0.0581	0.1749	16
CBA.AX	30	0.0413	0.1724	30	0.0425	0.1623	32
CBA.AX	60	0.0260	0.1159	60	0.0275	0.1053	62
CBA.AX	120	0.0173	0.0824	120	0.0196	0.0950	122
CBA.AX	240	0.0131	0.0634	240	0.0161	0.0797	242
TSLA	15	0.0661	0.2474	15	0.0651	0.2400	16
TSLA	30	0.0405	0.1665	30	0.0448	0.1592	32
TSLA	60	0.0276	0.1170	60	0.0295	0.1213	62
TSLA	120	0.0189	0.0860	120	0.0225	0.1101	122
TSLA	240	0.0134	0.0711	240	0.0168	0.0764	242
LLY	15	0.0369	0.1154	15	0.0372	0.1066	16
LLY	30	0.0216	0.0739	30	0.0225	0.0877	32
LLY	60	0.0167	0.0646	60	0.0183	0.0689	62
LLY	120	0.0114	0.0481	120	0.0136	0.0534	122
LLY	240	0.0081	0.0353	240	0.0106	0.0449	242

Table 1. RMSE (mean +- standard deviation over three seeds where the model is random) and mean maximum error, in the share's scaled price units, with the exact parameter count. For scale, the one-day-ahead naive forecast error of the same series is 0.0201 (CBA), 0.0255 (TSLA), 0.0148 (LLY).

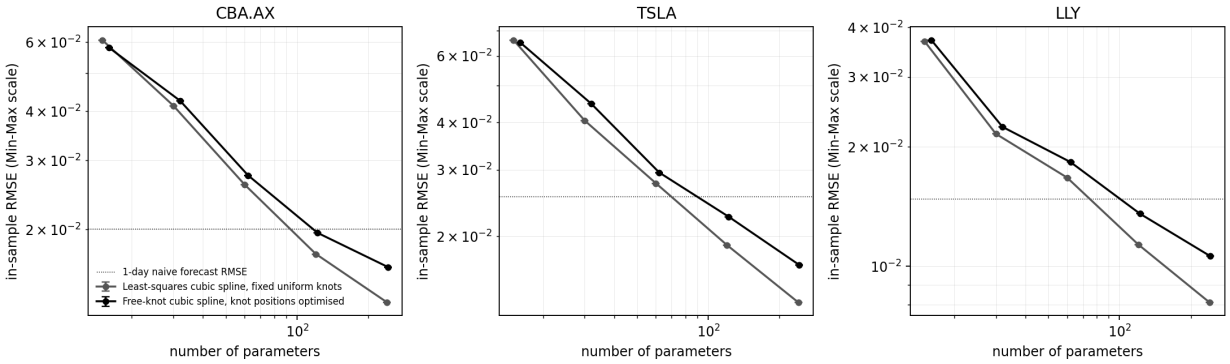


Figure 7. Error against parameter count for the two splines on the three shares (log-log). The dotted horizontal line is the one-day naive forecast error, for scale only.

Rates of improvement (slope of log RMSE against log parameters, budgets 30 to 240): fixed-knot -0.56, -0.53, -0.48 (mean -0.52); free-knot -0.48, -0.48, -0.38 (mean -0.45), for CBA, TSLA, LLY respectively.

7. Reading the results

Both splines work, and improve steadily. On CBA the fixed-knot spline's error falls from 0.0607 with 15 numbers to 0.0131 with 240; the worst single day falls from 0.2258 to 0.0634. By 240 parameters the typical error is below the day-to-day noise level of the series (0.020), i.e. the curve is tracking the price to within the size of an ordinary daily move.

Free knots do not pay for themselves on this data. At the same parameter count the free-knot spline is slightly worse on every share at every budget from 30 upward (CBA at 60: 0.0275 vs 0.0260; at 240: 0.0161 vs 0.0131), and its rate of improvement is lower (-0.45 vs -0.52). This is not because the knot search failed: given the *same number of intervals*, moving the knots does help (CBA, 30 intervals: RMSE 0.0353 fixed vs 0.0275 free). It is because each moved knot costs a parameter, and on a price line that parameter buys more when it is spent on an extra interval than on better placement. In the toy example of

Figure 6 the function had four special places and nothing else; a price line is rough everywhere, so there are no special places worth paying for. The knot-position maps in the result PDF show the optimiser clustering knots around the 2020 crash and the 2022 swings and leaving the rest coarse, which is the best it can do.

The rate is the one predicted by roughness. The fixed-knot spline improves with slope about -0.52, i.e. error proportional to 1 / sqrt(N). That is exactly the N^(-1/2) rate that Section 3.6 predicted for a function with Holder exponent 1/2. The spline is not being held back by anything about splines; it is being held back by the data.

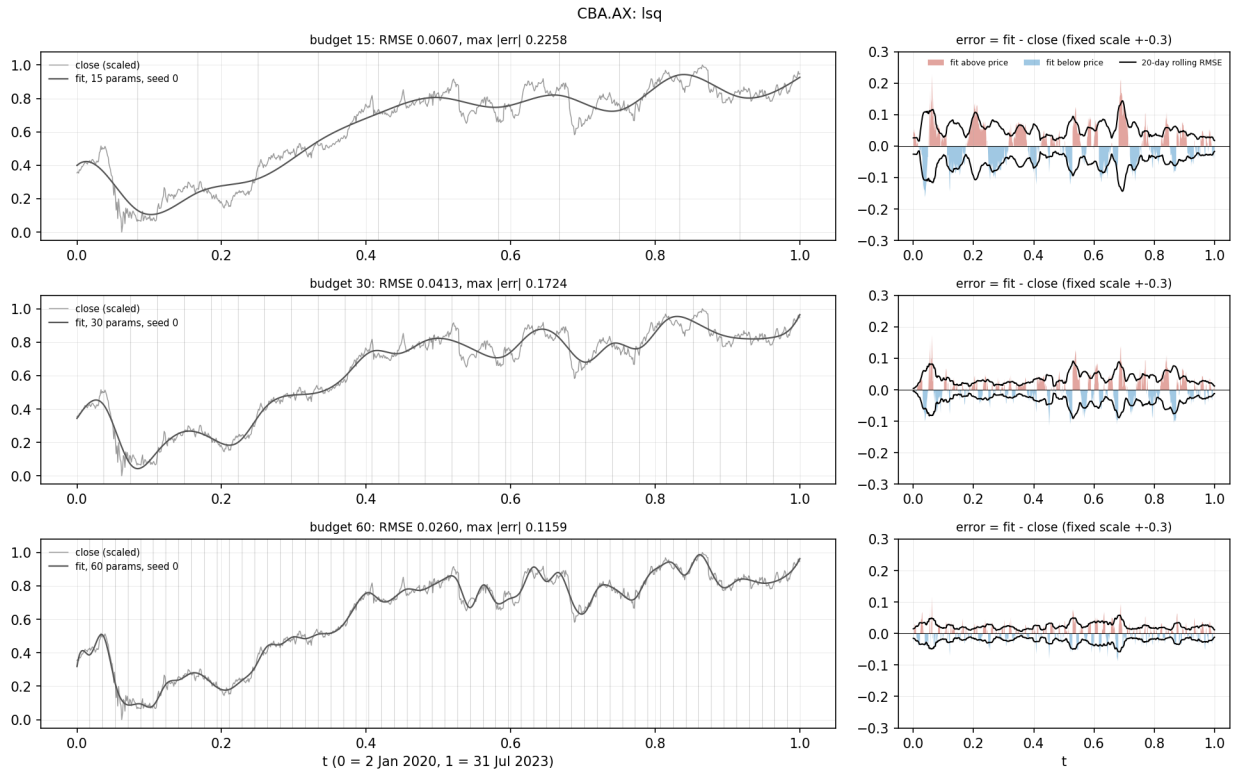


Figure 8. CBA, fixed-knot spline at 15, 30 and 60 parameters: the fit (left) and the error wave with its rolling-RMSE envelope (right). As the budget rises the envelope narrows and what remains is fine, fast jitter with no slow structure: the model has captured everything it can.

8. What it means

For someone wanting to describe a price history compactly, a plain cubic spline with evenly spaced knots is as good as the cleverer free-knot version, is instantaneous to compute, and has no randomness. The deeper lesson is that the error curve of the spline is a measurement of the data's roughness, and it comes out at the random-walk value. Any other method, however sophisticated, is competing against this benchmark; the other two documents in this series show that the best neural methods reach it and none beat it. Free-knot methods shine on functions with a few sharp features (the S1/S2 example in the earlier report, where they recovered the knots exactly); a price path is the opposite kind of function.

9. What this experiment cannot tell you (limitations)

• Everything is in-sample: the curve is scored on the very data it was fitted to. This is the right thing for an approximation study, but it says nothing about predicting prices that have not happened yet.

• Three shares and one period. The conclusions about the *models* are consistent across all three, but the numbers themselves would differ on other shares or periods.

• Parameter counts are matched approximately (within about 10 %), not exactly.

• Three random seeds; no formal significance test was applied. Where the spread across seeds is reported, treat differences smaller than the spread as not established.

• The free-knot search is non-convex; 12 starts were used, so 'did not win' means 'did not win with this optimiser'.

10. Glossary

term	meaning
Function	a rule turning an input (here, time t) into an output (here, a price).
Parameter	a number inside a formula that is chosen to make the formula fit the data.
Least squares	choosing the parameters so that the sum of squared errors is as small as possible.
RMSE	root-mean-square error: square the daily errors, average, take the square root; the typical error size.
Min-Max scaling	rescaling so the smallest value becomes 0 and the largest becomes 1.
Power law / rate	error proportional to N^(-a); a is the rate; on log-log axes a straight line of slope -a.
Holder exponent	a measure of roughness: how fast |f(x)-f(y)| shrinks as x and y approach; 1 = smooth, 1/2 = random-walk rough.
Seed	the starting value of the random number generator; different seeds give different random starts.
Spline	a curve made of polynomial pieces joined smoothly at knots.
Knot	a point where two polynomial pieces of a spline meet.
B-spline	one of the fixed 'bump' functions whose weighted sum gives a spline.
Coefficient	the weight of one bump; the spline's parameters.
Free-knot	a spline whose knot positions are optimised as well as its coefficients.
Non-convex	an optimisation landscape with many valleys, where a search can get stuck in the wrong one.